

# 챕터 1. MSA란 무엇인가?

밤 열한 시, 막 잠이 들려던 참에 휴대폰이 울렸습니다. 운영팀이었습니다.

**운영팀** : "사이트가 전부 멈췄습니다. 확인 좀 해 주세요."

잠이 확 달아났습니다. 노트북을 열자 대시보드가 온통 에러 로그로 덮여 있었습니다. 저녁 일곱 시에 시작한 할인 이벤트로 트래픽이 몰리면서, 그 부하를 버티지 못한 서버가 다운된 것입니다.

서둘러 서버를 재시작했지만 바로 안정되지 않았고, 트래픽이 빠지기 시작한 새벽 한 시가 되어서야 겨우 안정을 되찾았습니다.

트래픽 때문에 사이트 전체가 멈춰 버리다니.

다음 날 아침, 팀장이 자리로 왔습니다.

**팀장** : "이번처럼 트래픽 때문에 사이트 전체가 다운되면 곤란해요. 재발 방지 대책 좀 찾아봐요."

오픈이는 바로 답을 하지 못했습니다. 지난번에 트래픽이 몰렸을 때도 서버를 증설해 봤지만, 비용과 노력만 잔뜩 들었을 뿐 같은 장애가 되풀이됐습니다. 결국 선배를 찾아갔습니다.

**오픈이** : "선배님, 이번에 트래픽이 몰렸다고 사이트가 통째로 죽어버렸는데요. 이거 어떻게 해야 할까요?"

**선배** : "모든 기능이 한 서버에 다 뭉쳐 있어서 그래요. 한쪽에 부하가 걸리면 전체가 다 같이 멈추는 거죠. 이럴 때는 기능을 독립된 서비스로 분리해야 해요. 그러면 한 곳에 트래픽이 몰려 서버가 다운되더라도, 다른 기능은 멀쩡하거든요."

## 준비하기. 챕터 2부터 시작될 실습을 위해 미리 준비

이 챕터는 개념만 다루므로 직접 코드를 작성하지 않습니다. 챕터 2부터 실습이 시작되니, 그전에 도구 설치와 레포 위치를 미리 확인해 두세요.

### 1. 실습 환경

| 도구             | 사용<br>챕터 | 설치 주소                                                                                                         |
|----------------|----------|---------------------------------------------------------------------------------------------------------------|
| Docker Desktop | 챕터 2~    | <a href="https://www.docker.com/products/docker-desktop/">https://www.docker.com/products/docker-desktop/</a> |

| 도구                   | 사용<br>챕터 | 설치 주소                                                                     |
|----------------------|----------|---------------------------------------------------------------------------|
| Minikube             | 챕터 3~    | <a href="https://minikube.sigs.k8s.io/">https://minikube.sigs.k8s.io/</a> |
| Hoppscotch (브라우저 확장) | 챕터 2~    | <a href="https://hoppscotch.io/">https://hoppscotch.io/</a>               |

## 2. 챕터별 소스 코드

실습 코드는 두 레포로 제공됩니다.

| 레포    | 용도                           | 주소                                                                                          |
|-------|------------------------------|---------------------------------------------------------------------------------------------|
| start | 직접 작성하는 예제<br>(실습용). 클론해서 진행 | <a href="https://github.com/metacoding-12-msa/start">github.com/metacoding-12-msa/start</a> |
| final | 완성된 전체 코드. 막히면<br>참고         | <a href="https://github.com/metacoding-12-msa/final">github.com/metacoding-12-msa/final</a> |

두 레포 모두 챕터별 폴더로 구성됩니다.

| 챕터   | 폴더                   | 주제                         |
|------|----------------------|----------------------------|
| 챕터 2 | <a href="#">ex01</a> | 동기 REST + 보상 트랜잭션          |
| 챕터 3 | <a href="#">ex02</a> | DDD + 클린 아키텍처 + Kubernetes |
| 챕터 4 | <a href="#">ex03</a> | Kafka + Orchestration Saga |
| 챕터 5 | <a href="#">ex04</a> | WebSocket 실시간 알림           |

챕터 2부터 start 레포를 클론해 해당 폴더에서 실습하고, 막히면 final 레포의 같은 폴더에서 완성 코드를 확인하세요.

## 1.1 모놀리식 - 쇼핑몰을 하나의 서버로 만들면 어떻게 될까?

### 1.1.1 처음에는 아무 문제가 없었다

백화점을 떠올려 보세요. 수십 개의 매장이 한 건물 안에 모여 있습니다. 고객은 한 곳에서 모든 것을 해결할 수 있습니다. 백화점 입장에서는 전기·냉방·보안·고객 데이터를 한 곳에서 통합 관리할 수 있습니다. 이 구조는 단순하고 효율적입니다.



그림 1-1. 백화점 - 모든 매장이 한 건물에 모여 있는 구조

소프트웨어에서도 동일하게 적용됩니다. 처음에는 **하나의 서버에 모든 기능을 넣는 모놀리식(Monolithic) 구조**가 단순하고 관리가 편합니다.

### 1.1.2 성장하면서 균열이 생긴다

한 공간에 모든 매장이 모여 있는 백화점은 시간이 지나면서 문제가 보이기 시작합니다. 한 매장에 화재가 발생하면 전 층이 함께 대피해야 합니다. 또 정전이 발생하면 건물 전체가 함께 멈춥니다.

이 약점은 소프트웨어의 모놀리식에서도 그대로 나타납니다.



그림 1-2. 모놀리식 쇼핑몰 - 모든 기능이 하나의 서버에

모든 기능이 한 서버에서 함께 돌아가다 보니, **한 곳의 문제가 곧 전체의 문제가 됩니다.** 주문에 트래픽이 몰리면 회원·상품·배달까지 함께 느려지고, 특정 기능만 확장하고 싶어도 서버 전체를 확장해야 합니다. 또 작은 수정에도 전체를 재배포해야 합니다.

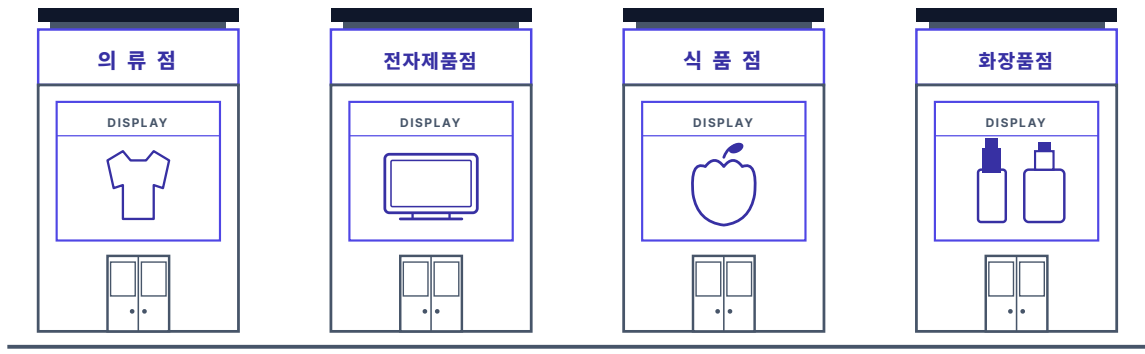
하나의 서버에 다 모여 있으니, 어젯밤 서버가 전부 같이 멈춘 거였구나.

## 1.2 마이크로서비스 - 역할을 나눈다

### 1.2.1 백화점 vs 개별 상점

개별 상점 방식은 백화점과 구조 자체가 다릅니다. 각 매장이 독립된 건물로 운영되어, 자신만의 전기·냉방·입구를 가집니다. 덕분에 한 매장에 화재가 발생하거나 문을 닫아도, 다른 매장은 그대로 영업할 수 있습니다.

### 개별 상점 — 매장마다 따로 선 건물



각자 건물·입구·운영을 따로 가진 네 개의 독립 상점

그림 1-3. 개별 상점 - 각 매장이 독립된 건물로 운영되는 구조

개별 상점처럼 하나의 큰 서버 대신 **기능별로 서비스를 분리한 구조가 MSA(Microservice Architecture)** 입니다.

### 마이크로서비스 — 서비스마다 따로 선 서버



서로 연결도 의존도 없이 각자 배포·확장되는 네 개의 독립 서버

그림 1-4. 마이크로서비스 쇼핑몰 - 기능별로 서비스를 분리

서비스를 분리하면 각 서비스는 독립적으로 배포하고, 독립적으로 확장할 수 있습니다. 그래서 주문 서비스에 문제가 발생해도 다른 서비스는 영향을 받지 않습니다.

## 1.3 시스템과 핵심 과제

문제를 이해했으니, 이제 만들어볼 시스템을 설계해 보겠습니다.

### 1.3.1 쇼핑몰 주문 시스템

이 책의 쇼핑몰 주문 시스템은 4개의 마이크로서비스로 구성됩니다.

| 서비스    | 포트   | 역할                  |
|--------|------|---------------------|
| 주문 서비스 | 8081 | 주문 생성·조회·취소 (핵심)    |
| 상품 서비스 | 8082 | 상품 목록, 재고 조회 및 증감   |
| 회원 서비스 | 8083 | 로그인, JWT 발급, 사용자 조회 |
| 배달 서비스 | 8084 | 배달 생성·조회·취소         |

주문 서비스를 중심으로 상품 서비스와 배달 서비스가 연결됩니다. 사용자가 주문하면 주문 서비스가 상품 서비스의 재고를 차감하고, 배달 서비스에 배달을 생성합니다.

### 1.3.2 서비스 간 요청의 두 가지 방식

각 서비스가 분리되면 서비스 사이의 통신이 별도로 필요합니다. 이 책에서는 두 가지 방식으로 구현합니다.

#### 방식 1. 직접 호출로 동기적 처리

첫 번째는 각 서비스가 다른 서비스를 직접 호출하는 방식입니다. 주문 서비스가 상품 서비스에 "재고 줄여줘"라고 요청한 후 응답이 올 때까지 기다리고, 응답이 돌아오면 다시 배달 서비스를 호출합니다.

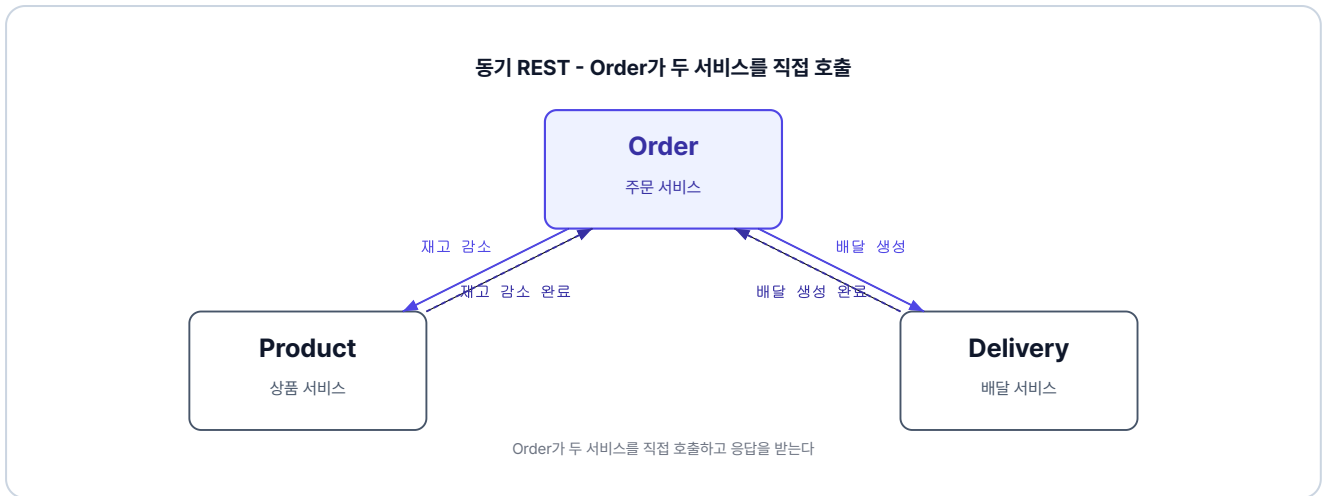


그림 1-5. 동기 REST - Order가 두 서비스를 직접 호출

이 방식은 구현이 단순합니다. 다만 호출한 서비스가 응답할 때까지 멈춰 있는 동기적 호출 방식이라, 한 단계가 지연되면 다음 단계도 지연됩니다.

## 방식 2. 메시지로 비동기 전환

다음은 서비스끼리 직접 호출하지 않고, **메시지로 요청을 주고받는 방식**입니다. 한 서비스가 메시지를 발행하면, 다른 서비스가 메시지를 받아서 처리합니다.



그림 1-6. 비동기 메시지 - 서비스를 분리하는 구조

발행한 서비스는 응답을 기다리지 않고 바로 다음 작업으로 넘어가므로, 한 서비스에서 응답이 지연되어도 다른 서비스는 지연되지 않습니다.

두 방식을 학습하며 각각의 장단점을 알아보겠습니다.

### 1.3.3 분산 트랜잭션 — 이 책의 핵심 과제

서비스를 분리하면 곧장 새로운 문제가 생깁니다. 모놀리식에서는 주문·재고·배달을 **하나의 트랜잭션으로 묶을 수 있기 때문에** 중간에 실패하면 전부 **자동 롤백**됩니다.

그런데 MSA에서는 각 서비스가 **독립된 데이터베이스**를 가집니다. 트랜잭션은 하나의 데이터베이스 안에서만 동작하므로, 서로 다른 DB에 걸친 작업은 하나의 트랜잭션으로 묶을 수 없습니다. 이렇게 여러 DB에 걸친 작업을 하나로 묶어야 하는 상황을 **분산 트랜잭션**이라고 합니다.

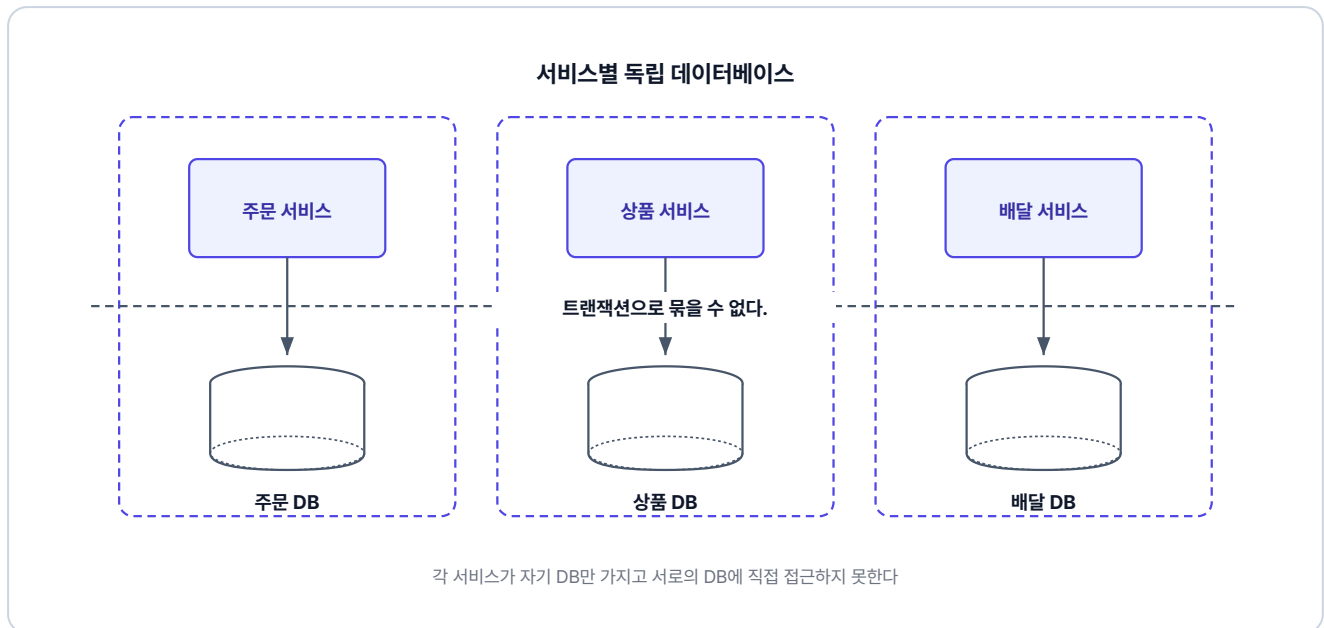


그림 1-7. 서비스별 독립 데이터베이스

**분산 트랜잭션(Distributed Transaction)이란?** 여러 독립된 데이터베이스에 걸친 작업을 하나의 논리적 단위로 처리해야 하는 상황입니다. MSA에서는 서비스마다 DB가 분리되어 있으므로 단일 트랜잭션이 불가능하고, 별도의 전략이 필요합니다.

**오픈이 :** "선배님, 재고는 줄였는데 배달이 실패하면 줄인 재고를 원복해야 하잖아요. 이런 걸 되돌리는 방법은 없나요?"

**선배 :** "자동으로 안 되니 직접 되돌려야 해요. 재고를 줄였으면 원복하고, 배달을 만들었으면 취소하고. 한 단계씩 거꾸로 되돌리는 거예요. 이걸 **보상 트랜잭션**이라고 해요."

**보상 트랜잭션(Compensating Transaction)이란?** 중간에 실패가 나면 이미 끝낸 작업을 역순으로 돌려 원래 상태로 되돌리는 방법입니다.

서비스를 분리하고, 분산 트랜잭션을 해결하는 것이 MSA의 과제입니다.

## 1.4 이 책의 학습 흐름

이 책은 하나의 시스템이 단계별로 진화하는 여정입니다. 각 챕터는 이전 챕터의 한계를 해결하는 방식으로 진행됩니다.



그림 1-8. 이 책의 학습 흐름

각 챕터에서 다루는 내용은 다음과 같습니다.

- 챕터 2에서는 각 서비스가 동기적으로 직접 호출하고, 중간에 실패하면 보상 트랜잭션으로 되돌립니다.
- 챕터 3에서는 도메인과 클린 아키텍처를 중심으로 서비스 구조를 다시 설계합니다.
- 챕터 4에서는 동기 방식을 메시지를 통한 비동기 방식으로 전환합니다.
- 챕터 5에서는 처리가 끝난 순간 사용자에게 실시간으로 알립니다.

이 책은 코드 작성보다 전체적인 개념과 흐름을 이해하고, 단계별로 실습하며 익히는 것을 목표로 하고 있습니다. 한 단계씩 학습하며 MSA의 큰 흐름을 따라가 보겠습니다.

## 이것만은 기억하자

- **모놀리식**은 처음에는 단순하지만, 서비스가 커지면 배포·장애·확장 문제가 생깁니다.
- **마이크로서비스**는 기능별로 서비스를 분리하여 각자 독립적으로 배포하고 확장할 수 있게 합니다.
- 각 서비스의 DB가 분리되어 있어 단일 트랜잭션으로 묶을 수 없습니다. 이것이 MSA의 핵심 과제인 **분산 트랜잭션**입니다.
- 분산 트랜잭션은 서비스끼리 **직접 호출·보상**하는 방식과, **메시지로 비동기 통신**하는 방식으로 학습합니다.